

Nama: Gilang lfdhaalih

Kelas: XI RPL

Absen: 18

KONSEP ENKAPSULASI

PERCOBAAN 1

Pertanyaan:

1. Adakah atribut dan method dari class Motor yang tidak bisa diakses dari dalam class itu sendiri? Sebutkan jika ada!

Jawab: Tidak ada, karena dari tiap atribut baik itu kodeMtr, jenisMtr, kecepatan dan gigi bisa diakses langsung didalam class itu sendiri tanpa adanya batasan.

2. Dari hasil percobaan tahap 3 (hasil compile dan run terhadap Motor.java), apakah Anda menemukan error? Jelaskan (error atau tidak error hasilnya) dikaitkan dengan modifier access dari atribut maupun methodnya!

Jawab: Tidak, saat class Motor dijalankan atribut yang ada didalamnya bisa diakses dan dijalankan dalam satu kelas yang sama. Terkait mengenai access modifier di dalam kode yang dimana terdiri dari public;private; dan default, yang dimana dari public bisa diakses dari class manapun, maupun dari package yang sama bahkan yang berbeda; default yang bisa diakses didalam kelas yang sama maupun beda dalam satu package yang sama; lalu private yang hanya bisa diakses di class yang sama

3. Dari hasil percobaan tahap 5 (hasil compile dan run terhadap MotorDemo.java), apakah Anda menemukan error? Jika iya, jelaskan apa penyebab terjadinya error tersebut!

Jawab: Ada, terdapat 2 error yang terjadi di atribut kecepatan dan gigi, hal dapat terjadi di MotorDemo dikarenakan access modifier dari kedua atribut tersebut memiliki sifat private yang berarti tidak bisa diakses di class lain

4. Atribut dan method apakah dari class Motor, yang hanya bisa diakses dari dalam class itu sendiri?

Jawab: atribut kecepatan dan gigi; serta method kurangiPorsneling dan tambahPorsneling

5. Atribut dan method apakah dari class Motor, yang bisa diakses dari class lain yang berada pada paket yang sama dengan class Motor tersebut?

Jawab: atribut: kodeMtr, merkMtr, jenisMtr. Method: start, tambahVelocity, kurangiVelocity, stop, dan tampilData.

6. Bisakah atribut jenis dari class Motor diakses dari class MotorDemo? Jika bisa, jelaskan lebih lanjut mengapa bisa?

Jawab: bisa, karena atribut tersebut bersifat (access modifier) default yang berarti atribut jenis bisa diakses di class yang berbeda tetapi dalam satu paket yang sama.

7. Jika di dalam method main di dalam class MotorDemo, ditambahkan sintaks s1.kurangiGigi(), apakah bisa? Jelaskan jawaban Anda!

Jawab: Tidak, karena method kurangiPorsneling memiliki access modifier private yang hanya bisa di akses di class itu sendiri/ class yang sama.

8. Perhatikan baris program 19 dan 24 dari class Motor, disitu terlihat dipanggil method kurangiGigi() dan tambahGigi() yang notabene memiliki access modifier private, dan tidak menimbulkan error. Mengapa bisa demikian?

Jawab: karena method tersebut di akses di satu class yang sama sehingga tidak menimbulkan error

9. Apa yang harus dilakukan agar error pada class MotorDemo bisa hilang?

Jawab: dengan mengubah access modifier di dalam atribut kecepatan dan gigi menjadi public

10. Berikan kesimpulan dari percobaan yang telah dilakukan dan beberapa pertanyaan terkait dengan access modifier!

Jawab: Kesimpulan: *Access modifier* (private, default, protected, dan public) berfungsi mengontrol tingkat keterlihatan dan keamanan data pada Java, di mana elemen ber-modifier private hanya dapat diakses dari dalam kelas itu sendiri, sedangkan modifier lainnya memungkinkan akses dari luar kelas sesuai lingkup hak aksesnya.

Percobaan 2

1. Mengapa pada baris pertama hasil keluaran muncul tulisan Objek terbentuk menggunakan konstruktor default ?

Jawab: Karena sintaks `Motor s1 = new Motor();` melakukan instansiasi objek `s1` dengan memanggil default constructor (konstruktor tanpa argumen). Kode didalam default konstruktor tersebut memiliki perintah `System.out.println` untuk menampilkan teks "Obyek terbentuk dari konstruktor default" saat objek selesai dibuat di memori.

2. Mengapa juga pada baris-baris setelahnya muncul tulisan Objek terbentuk menggunakan konstruktor berparameter 1 dan Objek terbentuk menggunakan konstruktor berparameter 2?

Jawab: Teks tersebut muncul karena terjadi peneraparan *constructor overloading*. Sintaks `Motor s2 = new Motor(2);` melakukan instansiasi dengan memanggil konstruktor yang memiliki 1 parameter, sedangkan `Motor s3 = new Motor(4, 25);` memanggil konstruktor dengan 2 parameter. Masing-masing blok kode konstruktor tersebut berisi perintah `System.out.println` yang mencetak teks ke konsol saat objek dibuat.

3. Perhatikan pada method `main()` dari class `Motor` tersebut, pada baris berapakah proses instansiasi yang menggunakan konstruktor berparameter dengan 2 parameter dijalankan?

Jawab: Proses instansiasi yang menggunakan konstruktor dengan 2 parameter dijalankan pada **baris ke-103** (`Motor s3 = new Motor(4, 25);`). Adapun pencetakan/penampilan data objeknya dilakukan pada baris ke-105 melalui pemanggilan method `s3.tampilData()`.

4. Perhatikan hasil keluaran program: Objek terbentuk menggunakan konstruktor berparameter 2
KODE = KOSONG MEREK = KOSONG JENIS = KOSONG KECEPATAN = 25.0 GIGI = 4 Setelah ditampilkan muncul nilai kode=KOSONG, merek=KOSONG, jenis=KOSONG, kecepatan=25.0 dan gigi=4. Dapat dari manakah nilai-nilai tersebut?

Jawab: Nilai-nilai tersebut diperoleh dari dua sumber saat proses instansiasi `Motor s3 = new Motor(4, 25);`:

1. **kecepatan = 25.0 dan gigi = 4**: Berasal dari nilai argumen yang dikirim ke konstruktor 2 parameter (`int g, float v`).
2. **merkMtr = "KOSONG"**: Berasal dari nilai awal (default) yang diisi secara eksplisit di dalam blok kode konstruktor 2 parameter.
3. **kodeMtr dan jenisMtr**: Bernilai kosong/default karena atribut tersebut tidak diinisialisasi di dalam konstruktor 2 parameter dan belum diisi nilai baru pada objek `s3`.

5. Misalkan di dalam class `Motor` ditambahkan konstruktor seperti di bawah ini: `public Motor(int x){ gigi = x; }` Bisa apa tidak? Jelaskan!

Jawab: **Tidak bisa (akan terjadi *compile error*).**

Penambahan konstruktor tersebut menyebabkan *duplicate constructor error*. Meskipun nama variabel parameternya berbeda (`g` dan `x`), keduanya memiliki **tipe data dan jumlah parameter yang sama** (sama-sama 1 parameter bertipe `int`).

Aturan *constructor overloading* pada Java mewajibkan setiap konstruktor memiliki perbedaan pada **jumlah parameter, tipe data parameter, atau urutan tipe data parameter (parameter signature)**.

6. Berikan contoh konstruktor berparameter dengan 3 parameter untuk class Motor tersebut, dan berikan juga contoh proses instansiasi yang menggunakan konstruktor dengan 3 parameter tersebut!

Jawab :

```
public Motor (int g, float v, String m){  
    System.out.println("Obyek terbentuk dari konstruktor berparameter 3");  
    merkMtr = m;  
    kecepatan = v;  
    gigi = g;  
}
```

Instansiasi:

```
Motor s4 = new Motor(5, 51, "Yahaha");  
s4.tampilData();
```

7. Berikan kesimpulan dari percobaan yang telah dilakukan dan beberapa pertanyaan terkait dengan konstruktor!

Jawab: **Kesimpulan:**

1. **Konstruktor** adalah *method* khusus yang dipanggil otomatis saat terjadi instansiasi objek (new). Nama konstruktor harus sama persis dengan nama *class* dan tidak memiliki *return type* (bahkan void).
2. Terdapat dua jenis konstruktor, yaitu **konstruktor default** (tanpa parameter) dan **konstruktor berparameter**.
3. Kelas Java mendukung **constructor overloading**, yaitu pembuatan lebih dari satu konstruktor dalam satu kelas. Syarat utamanya adalah setiap konstruktor harus memiliki **kombinasi parameter (signature) yang berbeda**, baik dari segi jumlah, tipe data, maupun urutan tipe data parameternya.

Percobaan 3

1. Sebutkan atribut dan method instansiasi yang dimiliki oleh class Motor!

Jawab:

Atribut Instans (*Instance Variables*):

- kodeMtr
- merkMtr
- jenisMtr
- kecepatan
- gigi

Method Instans (*Instance Methods*):

- start()
- kurangiPorsneling()
- tambahPorsneling()
- tambahVelocity(float v)
- kurangiVelocity(float v)
- stop()
- setKecepatan(float cc)
- tampilData()

2. Sebutkan atribut dan method class yang dimiliki oleh class Motor!

Jawab:

Atribut Class (*Static Variable*):

- sumTotalMotor (ditandai dengan kata kunci public static int)

Method Class (*Static Method*):

- sumTotalMotor() (ditandai dengan kata kunci public static void)
- main(String[] args) (juga merupakan static method bawaan Java)

3. Jelaskan perbedaan antara atribut dan method instansiasi dan atribut dan method class dari nilai yang disimpan di dalamnya!

Jawab: Perbedaan keduanya berdasarkan nilai yang disimpan adalah:

- **Atribut dan Method Instans (*Instance*):** Menyimpan nilai yang bersifat **independen/spesifik untuk masing-masing objek**. Nilai atribut pada satu objek bersifat mandiri dan tidak memengaruhi nilai atribut pada objek lainnya.
- **Atribut dan Method Class (*Static*):** Menyimpan nilai yang bersifat **terbagi (*shared*) untuk seluruh objek** dalam satu kelas. Semua objek berbagi satu nilai yang sama, sehingga jika terjadi perubahan nilai pada atribut *class*, perubahan tersebut akan berlaku untuk semua objek dari kelas tersebut.

4. Jelaskan perbedaan antara atribut dan method instansiasi dan atribut dan method class dari cara mengakses atribut dan method tersebut!

Jawab: Perbedaan cara mengaksesnya adalah:

- **Atribut dan Method Instans (*Instance*):** Harus diakses melalui **variabel objek** yang telah dibuat/di-instansiasi menggunakan perintah `new`.
 - *Contoh:* `s1.tampilData();` atau `s1.kodeMtr = "S001";`
- **Atribut dan Method Class (*Static*):** Dapat diakses secara langsung melalui **nama class-nya** tanpa perlu membuat objek terlebih dahulu.
 - *Contoh:* `Motor.sumTotalMotor();` atau `Motor.sumTotalMotor`

5. Perhatikan hasil keluaran di atas. Jelaskan bagaimana bisa muncul nilai 3 untuk jumlah total objek motor yang dibuat!

Jawab: Nilai 3 tersebut muncul karena adanya akumulasi pemanggilan *static method* `Motor.sumTotalMotor()`.

1. Atribut *static* `sumTotalMotor` memiliki nilai awal **0**.
2. Di dalam *method* `main()`, perintah `Motor.sumTotalMotor();` dipanggil sebanyak **3 kali** (masing-masing setelah instansiasi objek `s1`, `s2`, dan `s3`).
3. Setiap pemanggilan *method* tersebut mengeksekusi operasi `sumTotalMotor = sumTotalMotor + 1`, sehingga nilainya bertambah dari 0 menjadi 1, 2, dan akhirnya 3 saat dicetak ke konsol.

6. Berikan kesimpulan dari percobaan yang telah dilakukan dan beberapa pertanyaan terkait dengan konstruktor!

Jawab: Kesimpulan dari percobaan mengenai atribut & method *instans* dan *class* adalah:

1. **Ciri/Bentuk:** Anggota *class* (*static*) selalu ditandai dengan kata kunci `static`, sedangkan anggota *instans* tidak menggunakan kata kunci `static`.
2. **Cara Mengakses:**
 - **Instans:** Diakses melalui variabel objeknya (format: `namaObjek.namaAtribut` atau `namaObjek.namaMethod()`).
 - **Class:** Diakses langsung melalui nama kelasnya (format: `NamaClass.namaAtribut` atau `NamaClass.namaMethod()`).
3. **Penyimpanan Nilai:**
 - **Instans:** Menyimpan nilai secara **independen** (mandiri). Setiap objek memiliki nilainya sendiri, sehingga perubahan nilai pada satu objek tidak memengaruhi objek lain.
 - **Class:** Menyimpan nilai secara **bersama/terbagi (*shared*)**. Semua objek dari *class* yang sama menggunakan satu nilai variabel yang sama di memori.